

Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica

Departamento de Ingeniería en Computación

Semestre 2027-1



Asignatura:

Sistemas Operativos

Profesor:

Gunnar Eyal Wolf Iszaevich

Grupo: 7

Exposición: Sistemas operativos y computadoras cuánticas

Investigación sobre Diseño de Sistemas Operativos para procesadores cuánticos

---

Alumnos:

Hernández Andrade Miguel Ángel

Santos Cruz Lair Abraham

Fecha de entrega:

19 de octubre del 2026

Índice

1. Introducción **2**

2. NISQ **2**

3. Diseño de sistemas operativos para procesadores cuánticos **2**

4. Gestión de recursos **3**

4.1. Gestión de la memoria 3

4.2. Corrección de errores 4

4.3. Planificación de tareas 4

4.4. Integración de Sistemas Operativos tradicionales en procesadores cuánticos . 4

5. Gestión del hardware **5**

5.1. Modelo híbrido 5

5.2. Control a ultra-baja temperatura y abstracción 5

5.3. Pilas de software y lenguajes actuales 5

6. Conclusiones **6**

7. Referencias **6**

1. Introducción

La computación cuántica cambió el procesamiento de la información al sustituir el bit tradicional(0,1) por el qubit, aprovechando principios como la superposición y el entrelazamiento. Sin embargo, el hardware actual (era NISQ) enfrenta una barrera crítica: alta sensibilidad al ruido ambiental y tiempos de coherencia que apenas duran fracciones de segundo. Ante esta volatilidad, los sistemas operativos tradicionales colapsan. Al no estar diseñados para gestionar estados probabilísticos y efímeros, generan cuellos de botella masivos y pérdidas críticas de fidelidad en la ejecución. Para resolver este problema surge el Diseño de Sistemas Operativos Cuánticos (QOS). Un QOS actúa como un orquestador que sincroniza las CPUs clásicas con los procesadores cuánticos (QPUs), administrando la asignación espacio-temporal de los qubits, el agendamiento dinámico y la mitigación de errores en tiempo real, todo a un nivel de bajo nivel arquitectónico. En esta investigación, analizaremos la arquitectura profunda de los QOS para demostrar cómo esta capa de software es el puente indispensable que transformará el frágil hardware cuántico de hoy en el motor de cálculo del futuro.

2. NISQ

Noisy Intermediate-Scale Quantum, se utiliza para describir la etapa tecnológica exacta en la que se encuentra la computación cuántica.

- **N - Noisy:** Los procesadores cuánticos son extremadamente frágiles. Los qubits son sensibles a la más mínima perturbación externa. Esta sensibilidad genera "ruido" en el sistema, lo que provoca errores en las compuertas lógicas y decoherencia (La información cuántica dura apenas fracciones de segundo antes de colapsar).
- **IS - Intermediate-Scale:** Tenemos un hardware lo suficientemente potente para hacer cálculos complejos, pero no tenemos el volumen físico necesario para aplicar ajuste de errores. Como el hardware no puede corregirse a sí mismo por falta de qubits, toda la carga cae sobre el software, el cual tiene que inventar trucos algorítmicos para esquivar el ruido y lograr que el programa termine de ejecutarse antes de que los qubits fallen.

3. Diseño de sistemas operativos para procesadores cuánticos

Un sistema operativo cuántico funciona como la capa de abstracción principal entre los algoritmos de alto nivel y el hardware físico (QPU - Quantum Processing Unit). A diferencia

de un sistema operativo tradicional que administra recursos deterministas, un QOS (Sistema Operativo Cuántico) lidia con la inestabilidad de la era NISQ. Para lograr esta intermediación, el diseño del sistema operativo depende estructuralmente de una Capa de Abstracción de Hardware Cuántico (QHAL). De la misma manera en que un SO clásico oculta las diferencias subyacentes entre una arquitectura x86 y una ARM, la QHAL aísla al programador de la física del hardware. Gracias a esta capa, el sistema operativo puede procesar el mismo algoritmo lógico sin importar si la QPU está construida con circuitos superconductores, iones atrapados o topología de espín de silicio. Esta abstracción se materializa a través de la Arquitectura del Conjunto de Instrucciones Cuánticas (QISA), la interfaz directa entre el código compilado y el procesador. En su capa más profunda, el sistema operativo abandona el dominio lógico digital y entra en el plano analógico a través de la microarquitectura de control. Las instrucciones de máquina cuánticas no se traducen en variaciones de voltaje de 1 y 0. En su lugar, el QOS envía los comandos a los Generadores de Formas de Onda Arbitrarias (AWGs), traduciendo el código en envolventes de microondas calibradas o pulsos láser de altísima precisión. Esta etapa es la que convierte las matemáticas abstractas en estímulos físicos de radiofrecuencia diseñados milimétricamente para alterar el estado del qubit.

4. Gestión de recursos

4.1. Gestión de la memoria

La "memoria cuántica"presenta una diferencia fundamental frente a la RAM clásica: la información no se puede duplicar debido al Teorema de No Clonación.Por ello, la gestión de memoria en un QOS no consiste en copiar bloques de datos, sino en mapear estados en tiempo de vida útil. El sistema operativo administra registros de qubits virtuales y los mapea a qubits físicos dentro de la QPU. Debido a la decoherencia, el QOS rastrea continuamente el tiempo de decaimiento (T_1) y el tiempo de desfase (T_2) de cada qubit físico. Cuando un dato cuántico en memoria se acerca al límite de su coherencia, el sistema operativo debe liberarlo o refrescarlo mediante protocolos de almacenamiento en memoria cuántica de acceso aleatorio (qRAM), la cual emplea fotones u osciladores armónicos para almacenar estados cuánticos durante ciclos de procesamiento extendidos. * Teorema de no clonación: Es uno de los principios más restrictivos y fundamentales de la mecánica cuántica. Establece que es físicamente imposible crear una copia idéntica e independiente de un estado cuántico arbitrario y desconocido.

4.2. Corrección de errores

En el hardware de la era NISQ, los qubits interactúan con el entorno térmico y magnético, lo que genera errores de rotación de fase (phase-flip) y de inversión de bit (bit-flip). Dado que no es posible leer el qubit para verificar su estado, el QOS implementa esquemas de Corrección de Errores Cuánticos (QEC) a nivel de software y firmware.

El sistema operativo agrupa múltiples qubits físicos imperfectos para formar un único qubit lógico altamente estable (por ejemplo, utilizando el Código de Superficie). La función del QOS es coordinar la lectura continua de qubits auxiliares (llamados ancilla qubits). Estos qubits auxiliares miden los síndromes de error sin colapsar la información del qubit lógico. El SO procesa esos síndromes en microsegundos y aplica operaciones de corrección en tiempo real antes de que el error se propague por todo el circuito.

Código de Superficie: Es la arquitectura de corrección de errores cuánticos de tipo topológico más adoptada en la industria para procesadores de estado sólido (como los chips superconductores de Google o IBM). Su propósito es construir un solo qubit lógico altamente tolerante a fallos a partir de una matriz bidimensional (2D) de múltiples qubits físicos ruidosos.

4.3. Planificación de tareas

A diferencia de los planificadores clásicos (como Round Robin o First-Come, First-Served), el planificador de un QOS es consciente de la fidelidad y la topología. Como cada qubit físico de una QPU tiene una tasa de error diferente que varía hora tras hora, el planificador lee continuamente las matrices de calibración del hardware. Cuando un usuario envía una tarea, el QOS analiza la tasa de error de las compuertas (gate fidelity) y el nivel de interferencia cruzada (crosstalk). Con estos datos, asigna el trabajo dinámicamente a la región exacta del procesador que tenga el menor ruido en ese instante. Además, el planificador gestiona la multiprogramación espacial, dividiendo una sola QPU en particiones aisladas mediante "qubits de amortiguamiento"(buffer qubits) para ejecutar múltiples algoritmos de manera paralela y segura.

4.4. Integración de Sistemas Operativos tradicionales en procesadores cuánticos

Un procesador cuántico no funciona de forma autónoma; opera como un coprocesador (similar a una GPU) acelerado por una CPU clásica. Por esta razón, el diseño moderno de un QOS no busca reemplazar a Linux o Windows, sino integrarse con ellos mediante un

modelo híbrido.

El Sistema Operativo tradicional se ejecuta en los nodos de cómputo clásicos y maneja las llamadas al sistema (syscalls), la gestión de archivos, la autenticación de usuarios y la red. Cuando el sistema clásico detecta una subrutina cuántica, delega la ejecución al QOS a través de un controlador de bajo nivel (driver QPU). El QOS toma el control del bucle de retroalimentación de baja latencia entre la CPU y la QPU: envía los pulsos de control al procesador cuántico, recibe las lecturas binarias de las mediciones y devuelve los datos procesados al kernel del SO tradicional.

5. Gestión del hardware

5.1. Modelo híbrido

El paradigma dominante es la computación híbrida clásico-cuántica (e.g., algoritmos VQE o QAOA). En este modelo, el bucle de optimización clásico corre en la CPU/GPU dentro de la máquina clásica, mientras que las evaluaciones de funciones de onda se ejecutan en el procesador cuántico (QPU).

5.2. Control a ultra-baja temperatura y abstracción

Gran parte de los procesadores cuánticos de la actualidad (superconductores) operan a temperaturas mikelvin en criostatos de dilución. La capa física del SO cuántico abstrae las señales eléctricas analógicas de GHz enviadas a través de líneas criogénicas hacia las unidades de control del procesador.

5.3. Pilas de software y lenguajes actuales

Actualmente existen pilas de software que sirven de puente entre el lenguaje de alto nivel y el hardware cuántico:

- **Frameworks de desarrollo:** Qiskit (IBM), Cirq (Google), PennyLane.
- **Lenguajes e IRs intermedias:** OpenQASM, Q#, LLVM/QIR (Quantum Intermediate Representation).

6. Conclusiones

El desarrollo de sistemas operativos para procesadores cuánticos requiere superar barreras físicas como la decoherencia, la corrección activa de errores y la sincronización a escalas de nanosegundos. La tendencia actual no busca reemplazar al sistema operativo clásico, sino integrar la QPU como un acelerador especializado gestionado a través de modelos híbridos y pilas de software en constante evolución.

7. Referencias

1. Giortamis, E., Romão, F., Tornow, N., Bhatotia, P. (2024). QOS: A quantum operating system. arXiv.https://arxiv.org/abs/2406.19120

2. Bertels, K., Sarkar, A., Vlad, M. I., Ashraf, I., Almudever, C. G., Fu, X. (2020). A quantum computer architecture stack. IEEE Micro, 40(6), 54-66. https://doi.org/10.1109/MM.2020.300966

3. OpenQASM Association. (2023). *OpenQASM 3.0 Live Specification*. Recuperado de https://openqasm.com/

4. Zhang, X., Li, G., Guo, G. (2024). QOS: A quantum operating system. arXiv. https://arxiv.org/abs/2

5. Fu, X., Riesebos, L., Lao, L., Almudever, C. G., Sebastiano, F., Versluis, R., Kouwenhoven, L. P. (2019). A heterogeneous quantum computer architecture. Proceedings of the ACM International Conference on Computing Frontiers, 222–230. https://doi.org/10.1145/3310273.33